Pillowfort: Bounded State-Machine Analysis

Bounded coordination analysis · Protocol v4-CS1 · Evidence edition 2 · 9 November 2026

Status: the M1–M4 models and staged composition were checked from revision `c726c5eccafa` with TLC 2.19 (tla2tools v1.7.4). Nine finite safety configurations completed without an invariant violation; seven deliberately weakened configurations produced counterexamples. The host-preview readiness check maps to a separately reproduced and fixed relay scheduling race. This is bounded analysis of abstractions, not a cryptographic proof, implementation refinement, deadlock-freedom result, independent audit, or unbounded liveness proof. The companion description is Pillowfort: An MLS-Based Protocol for Ephemeral Rooms.

Executed analysis

Configuration	Generated states	Distinct states
M1 delivery	1,186	338
M2 admission	267	122
M3 resume	24	16
M4 succession	991	362
Composed M1, 12 steps	15,297	670
Composed M1+M2, 12 steps	39,141	3,976
Composed M1+M2+M3, 12 steps	40,268	19,336
Composed M1–M4, 12 steps	530,008	172,897
Composed M1–M4, backlog 2, 16 steps	3,968,326	1,202,973

Standalone models use at most two operation/KeyPackage tokens, backlog capacity two, three succession identities, and before/after deadline regions. Composition uses one operation token and the stated depth bound. Counts are per configuration; they must not be interpreted as disjoint implementation states.

The seven sensitivity checks weaken persistence ordering, connection-bound grants, host preview readiness, KeyPackage reuse prevention, snapshot reconciliation, transfer-target binding, and admission send interlocks. All produced invariant counterexamples. Only the host-preview check maps to the implementation bug reproduced during this evaluation; deliberate mutations alone are not evidence of implementation vulnerabilities.

Exact models, configurations, tool identity, commands and traces are in `evidence/models/` in the public evidence bundle. The technical appendix specifies concrete fields and transitions; the verification matrix distinguishes model coverage from implementation tests and unmodeled obligations. The reduced M3 model omits concrete historical-removal IDs and receive-before-close event scheduling; the composition does not establish consensus among arbitrary client host/epoch views.

Two concrete boundaries lie outside these reduced models: repeated active-lifecycle signals during restored applications, and serialization of explicit retirement after a succession policy. Real-MLS/relay regressions and browser executions cover those defects; neither is claimed as a TLC-discovered issue.

The remainder records the broader analysis plan and properties. A listed obligation is not automatically a proved property: consult the coverage matrix for the executed subset. Deadlock checking was explicitly disabled for these bounded-prefix safety runs; no unrestricted idle action is used to manufacture a liveness result.

1. Verification objective

Can admission, delivery, recovery, and host transfer interleave with crashes, retries, expiration, and disconnection without accepting the wrong identity, losing an acknowledged transition, or granting unauthorized authority?

Use **TLA+ with the TLC model checker** for this coordination layer. Keep cryptography abstract: valid signatures and MLS transitions are checked capabilities, not reimplementations of Ed25519 or MLS. This can find state-machine errors; it cannot establish cryptographic security or correctness of the browser, storage engine, or deployed implementation. TLA+ overview

Start with four small models, then compose them. Add removal barriers before making claims about the combined membership protocol. Avoid starting with the entire application or game engine.

2. Model boundaries and state

Keep **client volatile state**, **client committed state**, **relay volatile computation**, **relay committed state**, and **in-flight effects** separate. Otherwise a model can accidentally make a network send and two independent storage writes atomic, hiding the failures being investigated.

Model variable	Meaning and implementation anchor
`member[d]`	Relay lifecycle, admission phase, resume phase and credential binding; `SecureRelayMemberStateV4`
`roster[d]`, `epoch[d]`	Device d's authenticated MLS view; retain per-device views rather than assuming instant agreement
`hostRelay`, `hostView[d]`	Committed relay authority and each device's signed application view
`grant`	Current request/token/device/connection/order/expiry binding; `currentGrant`
`pending`	Pending application or commit and its exact message identity
`outbox[d]`, `rollback[d]`	Client operation, pre-transition state, and retained result context
`backlog[d]`	Ordered delivery entries, including results, authorizations, and historical removal markers
`resume[d]`	Owning request, reconciled snapshot, exact terminator, and completion status
`transfer`	Authorization ID, source host, target, offer message ID and expiry; `pendingHostTransfer`
`succession`	Current signed host policy, immutable admission order, attempted candidates, and exact outstanding offer authorization
`removal`	Device/admission binding and exact Remove commit; `pendingZombieRemovals`
`identity[d]`	Provisional, authentication-ambiguous, or established durable identity
`storeRevision[d]`, `lockOwner`	Compare-and-swap and room-scoped exclusive mutation ownership

Names above are **model abstractions**, not a new wire schema. Keep event IDs, envelope message IDs, request IDs, admission IDs, authorization IDs, room instances, and connection IDs distinct.

Represent signatures as immutable authenticated tuples. An adversary may replay tuples or sign for a device it controls; it cannot manufacture an honest device's signature. Represent an MLS membership transition by its bound predecessor/successor state and evidence, not just a Boolean "encrypted." Do not infer transcript agreement from valid cryptography.

3. M1 — ordered delivery and crash recovery

Conceptual operation states: idle → grant requested → grant issued → sender transition stored → frame pending → result retained → result stored by sender → ACK recorded → settled. Rejection takes a stored rollback path; request cancellation is a terminal result bound to that exact request.

These phases describe a distributed operation, not one global enum. Sender and relay can be at different phases while messages are in transit.

Actions: request/issue/expire grant; persist sender transition; transmit frame; persist relay pending state; automatic host preview; persist accept/reject result; emit delivery; persist sender result or rollback; emit/process exact ACK; cancel blocked order request; retry; disconnect; crash; restore.

Safety properties:

- **M1.1 — Exact grant:** when a frame consumes a grant, it matches the grant's room, request, token, device, connection and order and satisfies its expiry check. The pending/accepted operation retains that binding after consumption. At most one unconsumed current grant exists.
- **M1.2 — Persist before effects:** a transmitted mutation has the required committed sender state; relay delivery effects follow the corresponding relay write; an ACK follows the client write it acknowledges.
- **M1.3 — Stable outcome:** an accepted outcome is not silently changed to rejection on replay. Rejection does not roll back unrelated later accepted work.
- **M1.4 — Prefix ACK:** removing delivery backlog entries requires the exact acknowledged front prefix, not an arbitrary later ID.
- **M1.5 — No duplicate application:** repeating the same delivered operation cannot apply its application effect twice within the modeled retained-state lifetime.

First adversarial schedules: crash after sender storage but before send; lose the acceptance ACK; crash after relay storage but before effects; replay a grant from an old socket; lose an `order-cancelled` result; fail storage after result receipt; retry while a membership barrier blocks the request.

Source anchors: contract sections on causal ordering and persistence; relay grant/pending/backlog records; engine durable mutations and controller result handling. See the bundled source map.

4. M2 — admission and identity ambiguity

Conceptual admission states: no request → invitation-authenticated pending device → explicitly approved → Add committed → targeted Welcome → bootstrap applied → profile proof verified → active. Rejected, expired and cancelled attempts do not become active; retry requires the specified fresh-admission context.

Model identity persistence alongside admission: provisional → authentication-ambiguous before transmission → established after authoritative reconciliation. Do not model every disconnect as rejection.

Actions: submit bound Hello; authenticate invitation; verify exact binding; authorize the device; persist Add; deliver/process commit; persist targeted Welcome/bootstrap; send/verify profile proof; activate; expire/cancel; reload; recover ambiguous identity; attempt conflicting admission.

Safety properties:

- **M2.1 — Activation evidence:** activation requires host authorization of the exact device, a matching authenticated membership transition, and the corresponding verified joining proof.
- **M2.2 — Immutable binding:** no transition substitutes the room instance, credential, KeyPackage digest, target or admission ID under an earlier authorization.
- **M2.3 — Pending is not active:** pending admissions cannot send normal member traffic or use active-device resume.
- **M2.4 — Single use:** an admitted KeyPackage digest cannot be reused, including after retirement or retry. Exhaustion refuses admission rather than recycling history.
- **M2.5 — Preserve uncertainty:** a possibly accepted identity is not erased as "unsent," replaced automatically, or evicted to make space for another attempt.

Treat bootstrap redaction as an abstract projection that excludes pre-admission chat/drawing fields. Check the concrete projection separately with tests; a state-machine abstraction alone cannot prove that every serialized field is redacted.

First adversarial schedules: disconnect after invitation authentication; crash after Add but before Welcome; replay a proof for another admission; reload immediately after auth transmission; receive late rejection for an older attempt; saturate unresolved-identity storage; cancel after an attempt may already have reached the relay.

Source anchors: contract admission and durable-client-state sections; invitation/member bindings; controller admission and engine persistence handling.

5. M3 — exact resume and removal barriers

Conceptual states: disconnected → device-authenticated replay phase → backlog prefix applied → authority snapshot reconciled → exact terminator observed → completion sent → completion acknowledged → outbound enabled.

An empty backlog follows the same protocol, using its owning resume request as the terminator sentinel. Model snapshot reconciliation explicitly; merely receiving a snapshot is not success.

Actions: authenticate retained device; enter replay phase; deliver entry; persist entry; ACK prefix; deliver historical removal marker and matching Remove commit; reconcile roster/host snapshot; deliver `backlog-end`; send/accept completion; retry; process final encrypted frame followed by close.

Safety properties:

- **M3.1 — No early work:** ordinary outbound work remains blocked until exact resume completion is acknowledged.
- **M3.2 — Exact completion:** completion binds the current room, device, connection/request context and reconciled terminator; an old completion cannot release a new replay phase.
- **M3.3 — No relay-only authority:** a snapshot cannot invent MLS membership, erase a member without matching removal evidence, or replace signed host authority.
- **M3.4 — Historical causality:** the required removal marker precedes its associated commit and survives the retirement/replay lifecycle that still needs it.
- **M3.5 — Receive before close:** earlier received authenticated work is processed before a subsequent close is reconciled; close does not discard a final removal or retirement event.

First adversarial schedules: empty-backlog replay with a stale sentinel; disconnect between snapshot and terminator; deliver a Remove commit after pruning its marker; final self-removal immediately followed by socket close; recover with a stale local revision or lost Web Lock.

Source anchors: contract exact-resume and historical-removal sections; relay `resumePhase`, `resumeRequestId`, backlog, and removal records; serialized controller receive/close handling.

6. M4 — manual handoff and consent-based succession

Manual states: no offer → signed offer recorded → bound authorization retained → target authorization stored → signed acceptance pending → outgoing-host validation → relay switch committed → views reconcile.

Disconnection states: host policy committed → host disconnected within grace → recovery or grace expiry → old identity fenced → earliest eligible candidate offered the pillow → authorization persisted and delivery acknowledged → explicit acceptance or refusal → signed response verified. Acceptance commits authority and initiates former-host removal; refusal advances to the next eligible candidate. Exhaustion retires the room.

Use authenticated admission order. Reconnection does not change it. Only a current host-signed policy authorizes succession offers; room membership alone is insufficient. Unavailable candidates are skipped. Refusal or expiry advances to the next eligible, unattempted candidate. Every member may refuse; universal refusal collapses the fort. Delivery acknowledgment is not acceptance.

Actions: publish/replace signed policy; offer a manual target; disconnect/resume host; advance the 120-second grace boundary; fence expired identity; select the earliest eligible candidate; retain/deliver/store a 30-second authorization and ACK its delivery; explicitly accept or decline; sign and verify the exact response; commit authority only on acceptance; advance the offer on refusal; retire on exhaustion; deliver outcomes; overflow another member's backlog; complete exact MLS removal; replay a marker; deliver retirement lifecycle; publish a successor policy.

Safety properties:

- **M4.1 — Scoped authority:** a transfer has either a matching signed manual offer or a current signed succession policy, plus the intended admitted target's signed acceptance bound to the exact authorization.
- **M4.2 — Explicit consent:** neither disconnection, timeout, membership, delivery ACK, nor offer receipt establishes host authority. A fresh explicit acceptance is required. Refusal must preserve the refusing member's status and must never promote that member.
- **M4.3 — Persisted switch:** authority and capability consumption precede dependent delivery effects. Unrelated backlog overflow cannot revoke an accepted transfer.
- **M4.4 — Mode-specific removal:** manual handoff retains the old host as a member. Post-disconnection succession fences the expired identity and requires the corresponding exact MLS Remove; neither path creates a replacement room or identity.
- **M4.5 — Reconcile views:** a snapshot cannot substitute an unknown policy, target, capability, or host view for the client's authenticated state and exact in-flight acceptance.
- **M4.6 — Honest-relay priority:** selection uses the earliest eligible admitted device in the current policy, independent of UI sorting and reconnect time; expired attempts are not silently retried as new grants.
- **M4.7 — Tenure binding:** an earlier host policy cannot authorize succession after the host changes. Capability expiry, role changes, and old-host resume races cannot produce two accepted authorities at an honest relay.
- **M4.8 — Barrier completion:** a retirement lifecycle notification releases a replayed historical marker even when the former host was already retired. Ordinary work remains blocked until required removal is reconciled; only the exact authorized succession may bypass the barrier to restore its coordinator.

Do **not** require every device's host view to change simultaneously. Temporary differences while acceptance and results propagate are legitimate. Check one authoritative host at an honest relay and allowed per-client reconciliation states. Signatures cannot prove that another endpoint is offline.

First adversarial schedules: substitute target, policy, choice, or authorization; replay a prior-tenure policy; cross the grace boundary during resume; disconnect the first candidate; refuse then attempt stale acceptance; expire an offer before acceptance; lose a refusal result; let all members refuse; lose the accepted result; overflow another recipient during transfer delivery; replay a removal marker between the Remove ACK and retirement completion; crash before the successor refreshes its policy.

Source anchors: bundled `secureGameReducer.ts` succession cases; relay `continueHostSuccession`, `expireActiveDisconnects`, signed-response handling and retirement completion; engine delegation/context checks and durable controls; controller automatic policy/offer presentation and explicit acceptance/refusal actions.

7. Composition and assumptions

Compose in this order: **M1 → M1+M2 → M1+M2+M3 with removal → all four.** Each stage must preserve the earlier properties, not merely pass its own isolated checks.

Check that admission/removal/transfer barriers block or cancel conflicting work without losing the exact cancellation needed to release a client intent. Model storage writes and network effects as separate steps. A browser crash restores its committed state, not its pre-crash heap. A hosted-runtime loss destroys the memory-only identity; it does not restore browser-style durable crypto state.

Use two distinct fault profiles:

- **Coordination profile:** honest relay, honest participating devices, unreliable network, crashes, storage failure, retries and timeouts. Check safety across failure schedules.
- **Adversarial-delivery profile:** relay/network may withhold, replay, selectively deliver, or present inconsistent metadata. Check local authentication/binding rejection. Do not assert global consensus, complete transcript delivery, or availability; the protocol does not provide those guarantees.

For liveness, state assumptions separately: eventual storage success, sufficient capacity/fresh identifiers, fair processing, and timely delivery before relevant deadlines. Succession additionally requires honest relay eligibility decisions and a willing, available candidate covered by the current policy. An offline original host need not prevent progress; universal refusal deliberately terminates the room. Fairness alone cannot guarantee completion if every grant expires before use.

Check expected terminal/blocked conditions explicitly. Do not add an unrestricted idle action simply to make deadlock checking pass.

8. Bounds, counterexample checks, and completion criteria

Begin with a host, an existing member and a joiner; two connections per reconnecting identity; two room instances; a few distinct operation/admission tokens; and backlog capacities of one and two. Bound operation count and refuse fresh allocation on exhaustion. Never wrap sequence numbers or recycle identifiers to shrink the state space.

Represent time as relevant deadline regions—before, at, and after expiry—and preserve monotonic clock constraints. Add configurations with larger bounds, multiple queued operations, and competing admission/removal/handoff work. Small-model success is evidence within those configurations, not a proof for arbitrary room size or unbounded execution.

Check that the model is sensitive to the failures it claims to cover. Deliberately remove one safeguard at a time and require a counterexample: persist-before-ACK, exact grant binding, snapshot reconciliation, single-use KeyPackage tracking, or transfer-target binding. These are proposed model mutations, not known implementation vulnerabilities.

For each discovered trace:

1. Classify it as an implementation issue, modeling error, or documented limitation.
2. Map every model action to concrete code and storage/effect boundaries.
3. Replay a real bug as a deterministic implementation regression where feasible.
4. Fix the implementation and model, then rerun both the affected checks and composed configurations.

A completed analysis must publish: the source snapshot, readable specifications, exact TLC/tool versions and configurations, abstraction assumptions, checked bounds, named properties, state/transition counts, outcomes, counterexample traces and their disposition, and implementation mappings. Record excluded properties and intentional liveness limits alongside successes.

Keep the existing functional tests. Concrete regression and browser evidence supplements these models, but neither substitutes for the other. Cryptographic proof, independent audit, whole-profile rollback resistance, endpoint secrecy, full implementation refinement, and deployment correctness remain outside the executed bounded analysis.

Source map

- Secure-room contract
- Relay types and transitions
- Participant controller
- Participant engine
- Application reducer